

¡ESPECIAL DE ARQUITECTURA DE SOFTWARE!

EL GUARDIÁN DEL BACKEND: PATRONES Y CALIDAD DE DATOS

Una guía superheroica sobre Repository Pattern, Service Layer, Unit of Work y la Integridad de Base de Datos en Django & DRF.

 **PRESENTADO POR EL HÉROE DE LA ARQUITECTURA DIGITAL**

CAPÍTULO 1

PATRONES DE ACCESO A DATOS

Cómo estructurar la persistencia para aislar la lógica, evitar código espagueti y proteger el sistema ante peticiones masivas.

1. REPOSITORY PATTERN

PATRÓN DE ACCESO #1

🦇 EL GUARDIÁN DE LA CAPA:

"¡Encapsulamos las consultas a la base de datos! El Repository actúa como una fortaleza que separa el acceso a datos de la lógica del sistema."

🔑 Justificación Clave: Si la base de datos o el ORM cambian, la lógica de negocio ni se entera. Todas las consultas (SQL/ORM) quedan aisladas en repositorios dedicados.



2. SERVICE LAYER

BACKEND COMICS #1



PATRÓN DE ACCESO #2

⚡ EL HOMBRE DE ACERO:

"¡Centralizamos las reglas de negocio! La Capa de Servicio evita que la lógica se mezcle con las vistas o con las consultas directas a la BD."

🗨 Justificación Clave: Las vistas (controllers) solo reciben y responden. Los servicios orquestan los flujos y validaciones de negocio en un único lugar reutilizable.

3. UNIT OF WORK (TRANSACTION.ATOMIC)

PATRÓN DE ACCESO #3

EL GUARDIÁN TECNOLÓGICO:

"¡Todo o nada! Unit of Work con `transaction.atomic` ejecuta múltiples operaciones como una sola transacción indivisible."

 Justificación Clave: Si ocurre un error a mitad del proceso, realiza un rollback automático evitando registros corruptos o datos incompletos en la BD.



CAPÍTULO 2

BUENAS PRÁCTICAS DE CALIDAD

Garantizando la consistencia, veracidad e impenetrabilidad de los datos desde el esquema SQL hasta la API REST.



INTEGRIDAD REFERENCIAL

Garantiza que las relaciones entre usuarios, propiedades y entidades sean siempre válidas mediante llaves primarias (PK) y foráneas (FK).



RESTRICCIONES DE BD

Aplica cláusulas UNIQUE, NOT NULL y CHECK a nivel motor de base de datos para evitar datos duplicados, nulos o con valores fuera de rango.

VALIDACIÓN DE DATOS

Verifica que la información entrante sea correcta antes de persistirla, usando la potencia de Django y Django REST Framework (DRF).

MATRIZ COMPLETA DEL PROYECTO

CONCEPTO	CATEGORÍA	JUSTIFICACIÓN TÉCNICA EN EL PROYECTO
Repository Pattern	Acceso a Datos	Encapsula consultas SQL/ORM, desacoplando el motor de persistencia del resto de la app.
Service Layer	Acceso a Datos	Centraliza reglas de negocio puras, liberando a las vistas/controladores de responsabilidades.
Unit of Work	Acceso a Datos	Agrupar múltiples operaciones en <code>transaction.atomic</code> para evitar estados inconsistentes.
Integridad Referencial	Calidad de Datos	Valida vínculos lógicos (PK / FK) entre usuarios, inmuebles y registros del sistema.
Restricciones (BD)	Calidad de Datos	Impone candados rígidos (UNIQUE, NOT NULL, CHECK) a nivel almacenamiento físico.
Validación (Django/DRF)	Calidad de Datos	Filtra y sanitiza la información en los Serializers y Forms antes de tocar la base de datos.

¡MISION CUMPLIDA! ¿PREGUNTAS?

¡EL SISTEMA ESTÁ PROTEGIDO Y LISTO PARA ESCALAR!

Arquitectura robusta con patrones de diseño limpios y una base de datos a prueba de errores.

DJANGO REST FRAMEWORK | POSTGRESQL | CLEAN ARCHITECTURE | TRANSACCIONES ATÓMICAS

IMAGE SOURCES



https://media.craiyon.com/2025-05-12/ynS5cRuYSx-_IjxIfty8iw.webp

Source: www.craiyon.com



https://media.craiyon.com/2025-04-26/7XIha4VcRniYdiwcI8Co_g.webp

Source: www.craiyon.com



<https://media.gettyimages.com/id/2189326390/vector/superhero-holding-shield-while-hacker-using-computer.jpg?s=612x612&w=gi&k=20&c=lixPs1ZPzJjCWJjBtgKq9GPwLBVc6Fc8jggh365dy38=>

Source: www.gettyimages.ie